

Automatización de procesos con n8n para la extracción y gestión de datos públicos: Pipeline RPA Macroentorno Ecuador

Fernando Alfonso Ocampo Gonzaga
Universidad Técnica Particular de Loja
Prácticum 2.2 - Computación
Loja, Ecuador

Resumen—La Automatización Robótica de Procesos (RPA) permite convertir tareas digitales repetitivas en flujos controlados, verificables y documentados. Este artículo presenta una versión reducida del desarrollo del Pipeline RPA Macroentorno Ecuador implementado con n8n, Docker, Oracle Cloud y Telegram. La solución automatiza la consulta de fuentes públicas del Banco Central del Ecuador, INEC, Superintendencia de Compañías y MINEDUC, respaldando archivos crudos, transformando datos heterogéneos y cargando registros en una tabla consolidada. El proyecto se estructuró en cuatro workflows independientes según periodicidad: diario, mensual, trimestral y anual. Esta división permite reducir ejecuciones innecesarias, controlar el uso de memoria, facilitar la validación y mantener trazabilidad mediante archivos, Oracle, GitHub y notificaciones por Telegram.

Index Terms—RPA, n8n, Oracle Cloud, Docker, Telegram, ETL, C4, datos públicos, macroentorno Ecuador.

I. INTRODUCCIÓN

Las organizaciones que trabajan con información pública dependen de fuentes distribuidas en portales, APIs, archivos descargables y servicios web. Aunque esos datos están disponibles, su actualización manual implica revisar sitios institucionales, comprobar si existe una nueva versión, descargar archivos, renombrarlos, validar que no estén vacíos y comunicar resultados. Este tipo de operación es repetitiva, propensa a errores y difícil de auditar cuando no se registra cada ejecución.

En este contexto, la RPA se utiliza como una estrategia para automatizar tareas digitales basadas en reglas. En lugar de que un usuario repita la misma secuencia de acciones, un workflow puede ejecutar consultas HTTP, guardar archivos, transformar datos, registrar evidencia, cargar información y enviar notificaciones. n8n resulta adecuado para este escenario porque permite construir flujos visuales con nodos especializados y complementar la lógica con código cuando se requiere normalización o validación adicional.

El proyecto parte de un primer reto académico donde se integraron Open-Meteo, REST Countries y Open Library. Ese ejercicio permitió practicar activación manual, consumo de APIs, transformación con nodos Code y consolidación de ramas. Sin embargo, el reto final tuvo un alcance mayor: automatizar fuentes reales del macroentorno ecuatoriano y preparar salidas para un proceso de datos institucional.

El reto final se orientó al Tablero Macroentorno Ecuador. Para ello se consideraron fuentes del Banco Central del Ecuador, INEC, Superintendencia de Compañías y MINEDUC. Cada fuente posee frecuencia y formato distinto; por ello, la solución se diseñó como una arquitectura modular y no como un flujo lineal único. El objetivo fue documentar una base conceptual, arquitectónica y práctica para construir un pipeline RPA mantenible, verificable y escalable.

II. MARCO CONCEPTUAL

II-A. Automatización Robótica de Procesos

RPA se entiende como el uso de robots de software para ejecutar actividades digitales repetitivas, estructuradas y basadas en reglas. En este proyecto, el robot no simula solamente clics, sino que actúa como una cadena de procesamiento que recibe entradas, aplica reglas, produce salidas verificables y deja evidencia de ejecución. Su valor principal está en reducir la variabilidad de tareas manuales y generar trazabilidad.

RPA es adecuada cuando el proceso tiene repetición, reglas relativamente estables y posibilidad de verificación. Estas condiciones se cumplen en el reto final porque las fuentes públicas se revisan con periodicidades definidas, los archivos se guardan en rutas controladas y cada ejecución debe informar su resultado. La automatización no reemplaza el análisis del equipo de Datos; se concentra en entregar insumos confiables.

II-B. n8n, nodos y orquestación n8n fue seleccionado porque permite construir workflows mediante nodos conectados. Cada nodo cumple una responsabilidad concreta: activar el flujo, consultar una fuente, validar una condición, transformar datos, guardar archivos, cargar en Oracle o enviar una notificación. Esta separación favorece la modularidad y permite probar cada parte de forma independiente.

El Manual Trigger se utilizó para pruebas, mientras que el Schedule Trigger se diseñó para ejecuciones programadas. El nodo HTTP Request fue el componente de extracción; el nodo Code permitió normalizar campos, construir nombres de archivo, crear metadatos, validar respuestas, generar SQL por bloques y preparar mensajes de Telegram. Los nodos de

archivo conservaron evidencias crudas y los nodos Oracle materializaron la persistencia.

II-C. Docker, Oracle Cloud y C4

Docker se utilizó como entorno local de ejecución para n8n. Al trabajar dentro de un contenedor, las rutas de Windows no se interpretan directamente; por eso se montó la carpeta local datos_macroentorno hacia la ruta interna /home/node/datos_macroentorno. Esta configuración permitió que n8n guarde archivos crudos y que el usuario pueda revisarlos desde Windows.

Oracle Cloud Autonomous Database se empleó como destino de persistencia. La tabla TAB_CONSOLIDADO_MACRO permitió centralizar registros heterogéneos mediante campos comunes. El modelo C4 Nivel 2 se usó para documentar contenedores, sistemas externos y responsabilidades, evitando confundir arquitectura con una simple captura de nodos.

III. ESTADO DEL ARTE APLICADO

La literatura sobre RPA muestra que la automatización genera valor cuando se aplica a procesos repetitivos, de alto volumen y con reglas claras. Syed et al. describen que RPA mejora la eficiencia operativa, pero también requiere bases técnicas y organizativas para evitar automatizaciones frágiles. Wewerka y Reichert presentan RPA como automatización de procesos rutinarios basados en reglas, definición que se ajusta al pipeline propuesto.

Los enfoques de adopción de RPA recomiendan seleccionar procesos adecuados, documentar criterios de ejecución y controlar excepciones. Por ello, el reto final no se planteó como una automatización improvisada, sino como una arquitectura con disparadores, extracción HTTP, validación, repositorio crudo, normalización, logs, Oracle y notificación.

Desde la perspectiva de pipelines de datos, el proyecto comparte principios ETL: extraer desde fuentes públicas, validar integridad, normalizar información y preparar una carga posterior. La diferencia es que n8n coordina estas tareas con triggers, nodos visuales, código, archivos y credenciales sin construir una aplicación completa desde cero. La calidad del pipeline depende de controlar datos incompletos, estructuras inesperadas y fallos de procesamiento.

IV. DESCRIPCIÓN DEL RETO FINAL

El reto final consistió en implementar progresivamente un pipeline RPA para el Macroentorno Ecuador. El proceso consulta fuentes públicas, descarga archivos o respuestas, valida recursos, guarda evidencia, registra resultados y comunica el estado de ejecución.

Las fuentes consideradas pertenecen al BCE, INEC, Supercias y MINEDUC. El Banco Central aporta indicadores económicos como WTI, riesgo país, sector real y cuentas nacionales. INEC aporta datos laborales y censales. Supercias aporta ranking y

directorio de compañías. MINEDUC aporta registros administrativos educativos. La periodicidad se convirtió en criterio de diseño: diario, mensual, trimestral y anual o estático.

Cada fuente se trató como unidad de procesamiento con nombre lógico, institución, categoría, URL, formato esperado, carpeta destino, periodicidad y reglas de validación. Esta

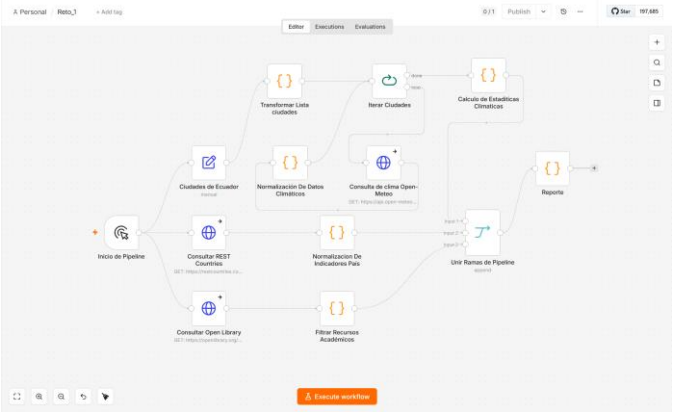


Figura 1: Workflow del primer reto en n8n usado como antecedente técnico.

Cuadro I: Workflows implementados por periodicidad.

Workflow	Periodo	Fuentes	Salida
Diario	Diaria	BCE WTI y Riesgo País	JSON/Oracle
Mensual	Mensual	BCE IEE y Supercias Directorio	XLSX/Oracle
Trimestral	Trimestral	INEC ENEMDU	ZIP/Oracle
Anual	Anual	Supercias, MINEDUC, INEC y BCE	CSV/XLSX

configuración permitió mantener un patrón común aunque las fuentes fueran heterogéneas.

V. ARQUITECTURA C4 NIVEL 2

La arquitectura se documentó mediante C4 Nivel 2 porque este nivel muestra contenedores y sistemas relacionados sin detallar cada nodo interno de n8n. En la vista general, el sistema principal es el Pipeline RPA Macroentorno Ecuador. Dentro del sistema se ubican triggers, orquestador n8n, extracción, validación, repositorio crudo, parsing, normalización, consolidación, manejo de errores y respaldo.

Fuera del sistema se representan las fuentes públicas, Oracle Database, el servicio de notificación y el equipo de Datos. Esta separación permite entender qué componentes pertenecen al pipeline y cuáles son dependencias externas. El operador puede activar manualmente el flujo durante pruebas y el planificador puede disparar ejecuciones automáticas por horario.

El módulo de extracción consulta URLs y descarga archivos o respuestas HTTP. La validación comprueba que exista archivo, que no esté vacío y que el formato sea coherente. Si la validación es positiva, el recurso se almacena como crudo; si

falla, se registra el incidente y se puede continuar con otras fuentes.

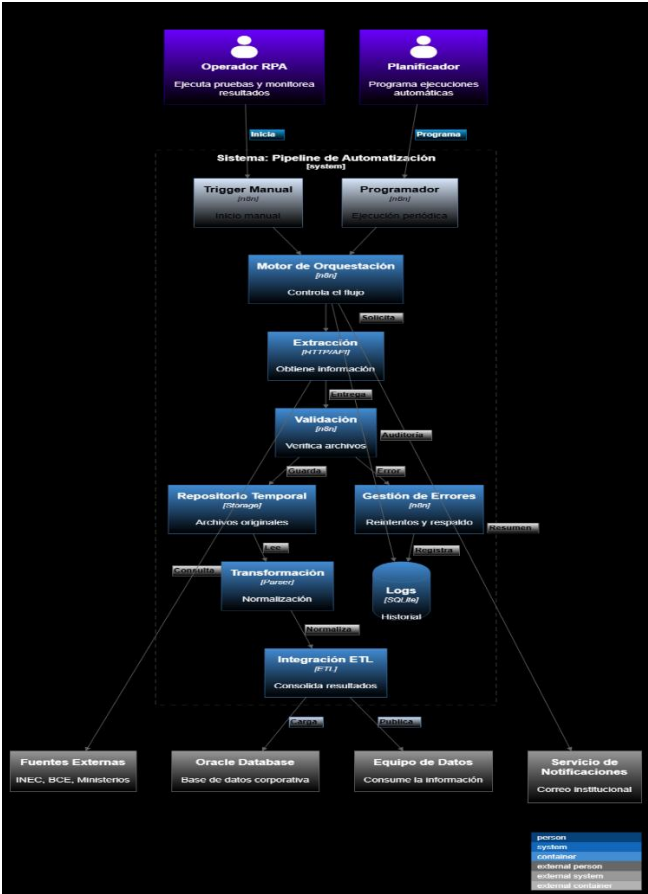


Figura 2: Arquitectura en C4 nivel 2

VI. CONFIGURACIÓN DEL ENTORNO

La ejecución se realizó en un entorno local con Docker Desktop. El contenedor de n8n se ejecutó en el puerto 5678, permitiendo abrir la interfaz desde <http://localhost:5678>. Docker fue un entorno de prueba y despliegue, no el requisito central de RPA; el mismo patrón podría ejecutarse en una instalación local o servidor si se configuran rutas y credenciales correctamente.

La estructura de carpetas se organizó por institución y tipo de fuente. En BCE se separaron

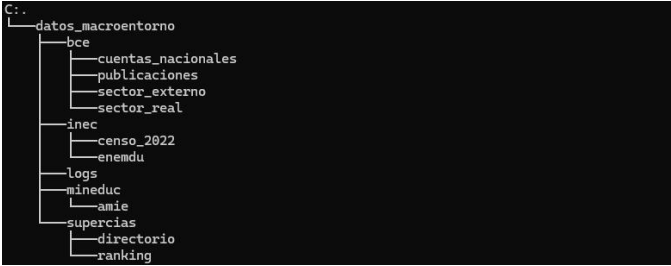


Figura 3: Estructura de carpetas para respaldo de archivos crudos. Cuadro

II: Campos principales de TAB_CONSOLIDADO_MACRO.

Campo	Función dentro del pipeline
INDICADOR	Identifica la fuente o indicador transformado.
FECHA_FISCAL	Representa fecha, periodo o corte del registro.
FECHA_EXTRACCION	Registra cuándo n8n procesó el dato.
ESTADO	Indica si la ejecución fue completa o incompleta.
NECESITA_RESPALDO	Marca registros que requieren revisión.
DETALLE_ERROR	Guarda información técnica cuando ocurre un fallo.
DATOS_JSON	Almacena el contenido normalizado de cada fuente.

sector_externo, sector_real, publicaciones y cuentas_nacionales. En INEC se separaron enemdu y censo_2022. En Supercias se separaron ranking y directorio. En MINEDUC se creó amie. Esta organización facilita ubicar evidencias crudas y defender el origen de cada archivo generado.

VII. ORACLE CLOUD Y PERSISTENCIA

La conexión entre n8n y Oracle Cloud fue uno de los elementos centrales. La base autónoma se configuró con el usuario ADMIN. La conexión se validó primero con SELECT

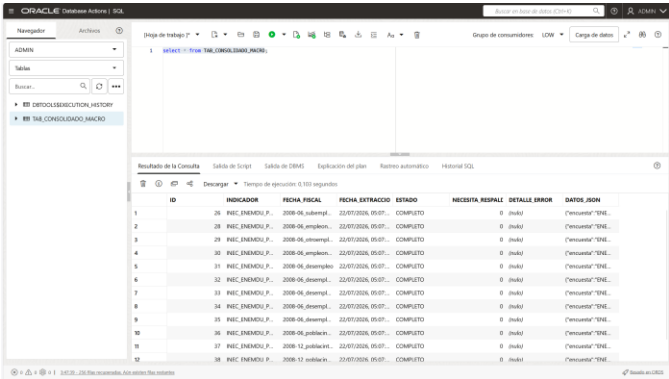


Figura 4: Validación de registros consolidados en Oracle Cloud.

SYSDATE FROM DUAL, consulta que permite comprobar si el nodo Oracle puede abrir sesión antes de insertar registros.

La conexión se resolvió mediante wallet porque la instancia usaba conexión segura. El wallet se montó dentro del contenedor en /home/node/oracle_wallet y se declaró la variable TNS_ADMIN. En n8n se activó Use SSL y se configuró el contenido del archivo ewallet.pem. Posteriormente, los nodos Oracle usaron la operación Insert or Update con coincidencia por INDICADOR y FECHA_FISCAL, evitando duplicados.

La tabla TAB_CONSOLIDADO_MACRO no reemplaza una bodega de datos completa, pero funciona como capa de consolidación ligera. Permite demostrar que el pipeline no solo descarga archivos, sino que produce información persistente y

consultable. El campo DATOS_JSON permite conservar particularidades de cada fuente sin crear una tabla distinta para cada caso durante la fase académica.

VIII. IMPLEMENTACIÓN DEL FLUJO DIARIO

El flujo diario se diseñó para fuentes que pueden actualizarse con mayor frecuencia. Incluye dos ramas: BCE WTI petróleo y BCE Riesgo País. Ambas ramas siguen la misma lógica: activar el flujo, consultar la fuente por HTTP, guardar una copia cruda en JSON, transformar el contenido y enviar registros a Oracle.

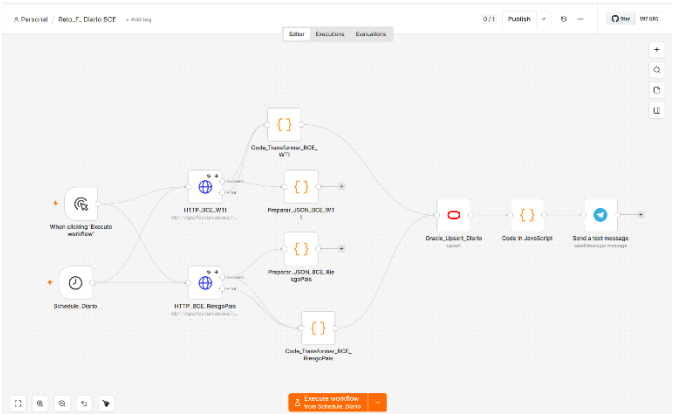


Figura 5: Workflow diario con carga en Oracle y notificación por Telegram.

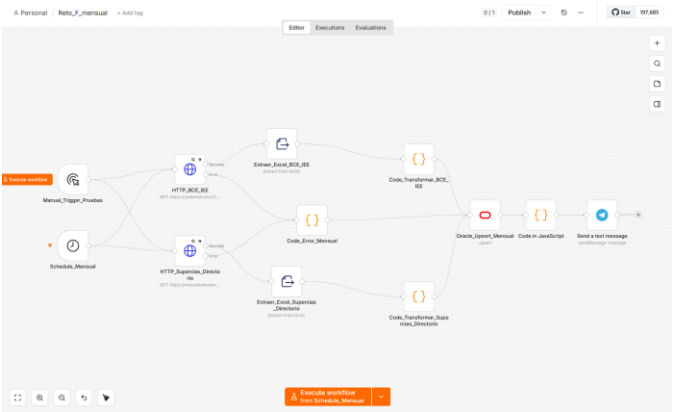


Figura 6: Workflow mensual con ramas BCE IEE, Supercias Directorio, Oracle y Telegram.

La rama de WTI consulta el recurso del Banco Central y deriva la salida hacia dos caminos: preparación y guardado de JSON crudo, y transformación para Oracle. La rama de Riesgo País replica el patrón, pero guarda la evidencia en la carpeta de publicaciones del BCE. Ambas ramas convergen en Oracle_Upsert_Diario. El resultado es doble: archivos crudos para trazabilidad y registros persistentes para consulta.

El cierre operativo se implementó con Telegram. Después de Oracle, el nodo Code construye un mensaje con estado, workflow, fuente, tabla y fecha. El nodo Telegram recibe chat_id y mensaje, y ejecuta el envío. Así, la ejecución diaria puede supervisarse sin ingresar manualmente a n8n.

IX. IMPLEMENTACIÓN DEL FLUJO MENSUAL

El flujo mensual agrupa fuentes que no requieren ejecución diaria: BCE Sector Real IEE y Supercias Directorio. Su objetivo fue demostrar que n8n puede procesar archivos descargables, extraer información desde hojas de cálculo o archivos estructurados y consolidar resultados en Oracle.

La rama BCE IEE descarga el archivo desde la fuente correspondiente y genera dos salidas: una hacia guardado crudo y otra hacia extracción del contenido. Posteriormente, el nodo Code define indicador, fecha fiscal, estado y datos normalizados. La rama Supercias Directorio sigue un patrón similar, pero por volumen de registros se incorporó carga controlada y generación de SQL por bloques.

La notificación mensual ayuda a cerrar el ciclo cuando se trabaja con archivos grandes. El mensaje resume si la carga fue completada, qué fuente fue procesada, qué tabla fue afectada Telegram.

Cuadro III: Secuencia técnica del flujo trimestral ENEMDU.		
Etap	Nodo principal	Decisión técnica
Consulta	HTTP INEC	Consultar página ENEMDU.
Búsqueda	Code ZIP	Detectar enlace vigente.
Descarga	HTTP ZIP	Obtener binario original.
Respaldo	Guardar ZIP	Conservar evidencia cruda.
Descompresión	Extract	Leer archivos internos.
Transformación	Code	Normalizar registros.
Persistencia	Oracle Upsert	Insertar o actualizar.

y qué operación se ejecutó. Esta evidencia complementa el historial de n8n y la consulta en Oracle.

X. IMPLEMENTACIÓN DEL FLUJO TRIMESTRAL

El flujo trimestral se enfocó en INEC ENEMDU. Esta implementación fue relevante porque incluyó búsqueda dinámica del archivo vigente, descarga de ZIP, guardado del archivo crudo, descompresión, transformación y carga en Oracle. La fuente no se consume como JSON simple, sino como paquete de archivos.

El flujo inicia consultando la página del INEC. Luego, un nodo Code analiza el contenido y obtiene la URL del ZIP vigente. Después, el nodo HTTP descarga el binario. Desde ese punto se separan dos caminos: guardar el ZIP original como evidencia y descomprimirlo para procesar los archivos internos.

Durante el desarrollo se identificó que el guardado del ZIP debía conectarse directamente al nodo de descarga, porque allí existe el binario data. Después de la descompresión, los archivos internos cambian de nombre a file_0, file_1 u otros campos. Esta corrección permitió guardar la evidencia original y transformar los archivos internos sin romper el flujo.

XI. IMPLEMENTACIÓN DEL FLUJO ANUAL

El flujo anual concentra las fuentes más pesadas y heterogéneas: Supercias Ranking, MINEDUC AMIE, INEC Censo 2022 y BCE Cuentas Nacionales. También considera información de PIB, empleo y valor agregado bruto. Debido al tamaño de algunos archivos, fue el workflow con mayor exigencia de memoria en el entorno local.

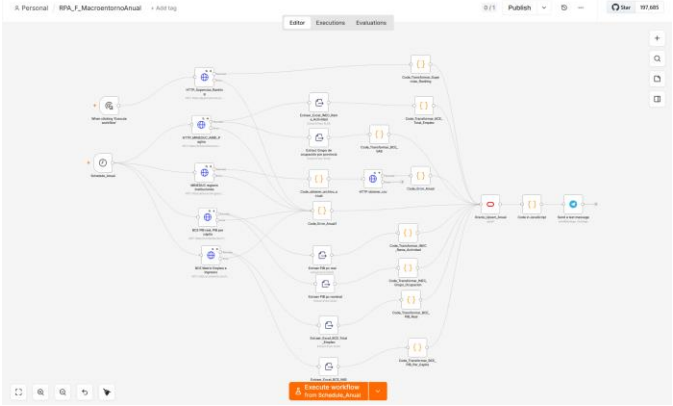


Figura 7: Workflow anual con fuentes Supercias, MINEDUC, INEC, BCE, Oracle y Telegram.

Cuadro IV: Ramas principales del flujo anual.

Rama	Fuente	Proceso aplicado
Supercias Ranking	Supercias	Descarga, respaldo y transformación.
MINEDUC AMIE	MINEDUC	Obtención dinámica de URL y descarga.
INEC 2022	Censo INEC	Extracción de hojas específicas.
BCE PIB	BCE	Lectura desde cuentas nacionales.
BCE MEI	BCE	Extracción de empleo y VAB.

Supercias Ranking se consultó mediante una rama de descarga, respaldo y transformación. MINEDUC AMIE requirió identificar dinámicamente el recurso vigente. INEC Censo 2022 trabajó con hojas específicas de un archivo grande. BCE Cuentas Nacionales procesó PIB real, PIB per cápita y matriz de empleo e ingresos. Cada rama mantiene el patrón: extraer, respaldar, leer, transformar y cargar.

El flujo anual se documentó como un workflow integrado, pero la validación práctica se ejecutó por ramas. Esta decisión no indica que el flujo esté incompleto; responde a una limitación del entorno local y al volumen de datos. En un despliegue productivo, las ramas anuales podrían separarse en subworkflows o ejecutarse con mayor memoria.

XII. VALIDACIÓN Y RESULTADOS

El primer resultado fue la construcción de cuatro workflows independientes en n8n. Esta división permite ejecutar cada grupo de fuentes según su frecuencia real de actualización. El segundo resultado fue la estructura datos_macroentorno, que

conserva evidencia cruda por institución y categoría. El tercer resultado fue la conexión funcional con Oracle Cloud y la carga en TAB_CONSOLIDADO_MACRO. El cuarto resultado fue el versionamiento de workflows en GitHub, excluyendo credenciales y archivos pesados.

La validación del flujo diario consistió en revisar archivos JSON del BCE y registros con indicadores WTI y Riesgo País. El flujo mensual se validó revisando respaldos de BCE IEE y Supercias Directorio. El flujo trimestral se validó verificando que ENEMDU descargue el ZIP, lo guarde, lo descomprima y

Cuadro V: Resumen operativo de ejecución y volumen.

Flujo	Espera	Ejecución	Filas aprox.	Volumen
Diario	Diario	12-18 min	11.133	Bajo/JSON
Mensual	Mensual	45-60 min	224.632	Medio-alto
Trimestral	3 meses	25-35 min	1.768	ZIP/CSV
Anual	Anual	2h-2h30	1.713.304	Alto/SQL

genere registros. El flujo anual se validó por ramas para evitar saturación de memoria.

El criterio de éxito fue común: que la fuente responda, que el archivo crudo se guarde en la carpeta correcta, que la transformación produzca campos comunes y que Oracle reciba los datos sin duplicar indicadores existentes. Este patrón confirma que la arquitectura C4 se materializó en una implementación funcional.

XIII. TELEGRAM, SQL Y EVIDENCIAS

Telegram se incorporó como cierre operativo del pipeline. La implementación usa dos nodos: un nodo Code que construye el mensaje y un nodo Telegram que lo envía. Esta separación mejora la mantenibilidad porque se puede modificar el texto del reporte sin tocar la credencial ni la operación de envío.

El nodo Code prepara principalmente chat_id y mensaje. El chat_id identifica el chat destino y se obtiene mediante la API getUpdates después de enviar un mensaje inicial al bot. El mensaje incluye workflow, fuente procesada, estado, tabla destino, fecha y resultado. En el nodo Telegram se usan expresiones como Chat ID = {{\$json.chat_id}} y Text = {{\$json.mensaje}}.

También se generó un archivo SQL como evidencia. Este script contiene el DDL o referencia de la tabla TAB_CONSOLIDADO_MACRO y consultas de validación para revisar registros por indicador, periodos cargados y posibles duplicados. La exportación se puede obtener con DBMS_METADATA.GET_DDL. Esta evidencia complementa los JSON de n8n, capturas, estructura de carpetas, Oracle y Telegram.

XIV. DISCUSIÓN TÉCNICA

La solución cumple porque automatiza extracción, respaldo, transformación, persistencia y notificación. La división por periodicidad evita una automatización ineficiente y facilita

explicar el proyecto durante la defensa. El flujo diario demuestra actualización frecuente; el mensual demuestra manejo de archivos periódicos; el trimestral demuestra ZIP y búsqueda dinámica; el anual demuestra integración de fuentes institucionales pesadas.

Oracle Cloud aporta un componente institucional porque los datos no quedan solo como archivos descargados. La tabla consolidada permite consultar registros y aplicar una operación de actualización controlada mediante INDICADOR y FECHA_FISCAL. GitHub aporta versionamiento de workflows, siempre que se excluyan wallets, contraseñas y archivos pesados.

Los problemas encontrados fortalecieron la solución: rutas de Docker, campos binarios, wallet Oracle, contraseñas, memoria y GitHub. Cada incidente obligó a aplicar criterio de ingeniería. Como mejora futura, se recomienda construir subworkflows reutilizables para logs, errores, nombres de archivo, carga a Oracle y notificación.

XV. MÉTODO DE TRABAJO INCREMENTAL

El método de trabajo fue incremental porque el reto integraba fuentes con formatos y volúmenes diferentes. En lugar de automatizar todas las instituciones al mismo tiempo, primero se validó el patrón base de ejecución: activar un flujo, consultar una fuente, guardar el recurso crudo, transformar la respuesta y cargar el resultado en Oracle. Esta forma de trabajo permitió reducir el riesgo y detectar problemas antes de que los workflows crecieran en complejidad.

La primera fase consistió en analizar las fuentes y clasificar sus periodicidades. En esta revisión se identificaron URLs, formatos de descarga, frecuencia de actualización, institución responsable y nivel de dificultad. La salida de esta fase fue una matriz de fuentes que permitió separar los procesos en cuatro grupos: diario, mensual, trimestral y anual. Esta clasificación se trasladó después a los Schedule Trigger de n8n.

La segunda fase fue el diseño arquitectónico. Se utilizó C4 Nivel 2 para identificar contenedores, sistemas externos y responsabilidades. Esta fase fue importante porque permitió distinguir entre el flujo visual de n8n y la arquitectura del sistema. El diagrama no representa cada nodo, sino los contenedores que cumplen funciones: activación, orquestación, extracción, validación, repositorio, normalización, persistencia y notificación.

La tercera fase consistió en construir los workflows. Primero se probaron ramas sencillas con respuestas JSON, luego ramas con archivos descargables y finalmente escenarios más complejos con ZIP y hojas de cálculo. Cada ejecución manual permitió revisar outputs, binary fields, rutas internas y estructura de registros. Esta validación por pasos fue necesaria para evitar errores encadenados.

La cuarta fase incorporó Oracle Cloud. Se configuró la base autónoma, se montó el wallet y se validó la credencial con una

consulta simple. Luego se creó la tabla consolidada y se conectaron los nodos Oracle de cada workflow. La quinta fase agregó Telegram como cierre operativo. Finalmente, se exportaron JSON, capturas, script SQL y evidencias para la entrega.

XVI. DETALLE OPERATIVO DE LOS WORKFLOWS

El flujo diario debe entenderse como el módulo de actualización frecuente. Su construcción parte de indicadores que pueden cambiar en periodos cortos y que son livianos de consultar. El Schedule Trigger puede ejecutarse todos los días y el Manual Trigger permite pruebas sin esperar la programación. La salida HTTP se divide en respaldo crudo y transformación para Oracle. Esta doble salida permite defender que el pipeline conserva la fuente original y, a la vez, genera un dato estructurado.

Cuadro VI: Fases metodológicas aplicadas.		
Fase	Actividad	Evidencia
1	Análisis de fuentes	Inventario por institución y periodicidad.
2	Diseño C4	Diagrama de contenedores del pipeline.
3	Workflows n8n	JSON exportados y capturas.
4	Oracle Cloud	Tabla, wallet, consulta y Upsert.
5	Telegram	Mensajes automáticos por flujo.
6	Documentación	Informe, script SQL y repositorio.

En el caso de WTI, el nodo HTTP consulta el recurso del BCE y entrega la respuesta al nodo de preparación JSON. El archivo se guarda con fecha para evitar sobrescrituras. En paralelo, un nodo Code extrae el valor relevante, define el indicador, construye la fecha fiscal y arma el objeto común. Riesgo País usa el mismo patrón, demostrando que la lógica se puede reutilizar con otra fuente diaria.

El flujo mensual responde a una necesidad diferente. BCE IEE y Supercias Directorio no requieren revisión diaria, pero sí necesitan respaldo y procesamiento periódico. Este workflow demuestra que el pipeline no se limita a APIs o JSON, sino que puede trabajar con archivos estructurados. La etapa de extracción convierte archivos descargados en items de n8n, y la etapa Code prepara la estructura común de persistencia.

Supercias Directorio tuvo una consideración especial por volumen. En lugar de depender solamente de una carga visual estándar, se preparó una estrategia de SQL por bloques. Esta decisión permite controlar mejor la inserción de registros y evita saturar la memoria cuando el archivo produce muchos items. En una defensa, esta rama evidencia que el flujo mensual maneja tanto fuentes medianas como fuentes de mayor volumen.

El flujo trimestral es más complejo porque no se limita a descargar una URL fija. La fuente ENEMDU se publica como

recurso cambiante; por ello, primero se consulta la página del INEC y luego un nodo Code identifica el enlace del ZIP vigente. Esta búsqueda dinámica es una mejora importante frente a un enfoque manual, ya que reduce dependencia de nombres de archivo que pueden cambiar.

Después de la descarga, el ZIP se guarda como evidencia cruda y se descomprime para procesar los archivos internos. El error de binary field enseñó que el guardado del ZIP debe hacerse antes de la descompresión, porque allí existe el campo binario original. Esta corrección es importante porque muestra que el workflow no solo fue construido, sino depurado con criterio técnico.

El flujo anual integra fuentes pesadas. Aunque se documenta como un workflow general, su ejecución local se valida por ramas. Esta decisión permite controlar memoria y tiempo de ejecución. Supercias Ranking, MINEDUC AMIE, INEC Censo y BCE Cuentas Nacionales tienen estructuras distintas, por lo que cada rama aplica su propia extracción, pero todas convergen en una salida común hacia Oracle y Telegram.

Cuadro VII: Procedimiento de reproducción.

Paso	Acción	Evidencia
1	Preparar carpeta y estructura de datos.	Captura de carpetas.
2	Ejecutar n8n en Docker.	docker ps.
3	Montar wallet y declarar TNS_ADMIN.	tnsnames.ora visible.
4	Configurar Oracle en n8n.	SELECT SYSDATE.
5	Crear tabla consolidada.	Tabla en Oracle Cloud.
6	Importar workflows JSON.	Cuatro flujos visibles.
7	Probar manualmente.	Ejecuciones correctas.
8	Verificar archivos crudos.	Evidencias en carpetas.
9	Verificar base de datos.	SELECT por indicador.
10	Exportar y entregar.	GitHub limpio.

XVII. PROCEDIMIENTO PARA REPRODUCIR LA IMPLEMENTACIÓN

Para reproducir la solución se debe preparar primero la carpeta del reto final. Dentro de ella se conserva datos_macroentorno, los workflows JSON, diagramas, capturas y documentación. No se debe incluir el wallet de Oracle ni archivos de credenciales. Esta separación evita exponer información sensible y permite entregar únicamente los elementos evaluables.

El segundo paso consiste en ejecutar n8n mediante Docker. El contenedor debe publicar el puerto 5678 y usar un volumen persistente para conservar configuración. Además, la carpeta datos_macroentorno debe montarse hacia /home/node/datos_macroentorno. Esta ruta interna debe usarse en todos los nodos de escritura. Intentar escribir directamente en rutas de Windows genera errores porque el proceso se ejecuta dentro del contenedor.

El tercer paso es montar el wallet de Oracle. La carpeta descargada desde Oracle Cloud se enlaza a /home/node/oracle_wallet y se define TNS_ADMIN con esa

ruta. Esto permite que el cliente Oracle usado por n8n encuentre tnsnames.ora y los archivos de seguridad. Luego se crea la credencial Oracle con usuario ADMIN, alias rpamacro_tp, SSL activado y contenido PEM del wallet.

El cuarto paso es validar Oracle con SELECT SYSDATE FROM DUAL. Si esta consulta funciona, se puede crear o verificar TAB_CONSOLIDADO_MACRO. El quinto paso es importar los workflows JSON. Cada flujo debe revisarse para confirmar credenciales, rutas, triggers y nombres de nodos. El sexto paso es probar manualmente cada rama antes de dejar activada la programación.

XVIII. EVIDENCIAS GENERADAS

La evidencia principal son los cuatro workflows exportados en formato JSON. Estos archivos permiten reconstruir la automatización en otra instancia de n8n. Las capturas de los flujos permiten revisar visualmente la arquitectura implementada. La carpeta datos_macroentorno demuestra que el pipeline conserva respaldo crudo y no depende únicamente de la carga en Oracle.

Oracle Cloud demuestra persistencia en la nube. La tabla TAB_CONSOLIDADO_MACRO contiene registros norma-

Cuadro VIII: Evidencias recomendadas para la entrega final.

Evidencia	Qué muestra	Valor
Flujo diario	WTI, Riesgo País, JSON y Oracle.	Fuentes diarias.
Flujo mensual	BCE IEE, Supercias y Oracle.	Fuentes mensuales.
Flujo trimestral	ENEMDU, ZIP y Oracle.	Fuente trimestral.
Flujo anual	Ramas anuales integradas.	Alcance completo.
Oracle Cloud	Tabla con registros.	Persistencia.
Carpetas	Archivos crudos por institución.	Trazabilidad.
GitHub	JSON y evidencias sin wallet.	Entrega limpia.

lizados y permite validar cargas mediante consultas SQL. Telegram demuestra observabilidad operativa porque permite recibir un resumen cuando un workflow termina. GitHub demuestra versionamiento siempre que el repositorio contenga JSON, diagramas y capturas sin incluir secretos.

XIX. SEGURIDAD Y VERSIONAMIENTO

La seguridad fue un aspecto importante porque la conexión a Oracle usa wallet y credenciales. El documento puede explicar que se usó Oracle Wallet, TNS_ADMIN y credenciales internas de n8n, pero no debe mostrar contraseñas, contenido PEM completo ni archivos del wallet. Estos elementos se mantienen fuera de GitHub y fuera de capturas públicas.

El repositorio debe contener workflows JSON, diagramas, capturas y estructura de carpetas. No deben subirse archivos .pem, .p12, .sso, .jks, tnsnames.ora, sqlnet.ora, ZIP de wallet ni bases crudas demasiado pesadas. La configuración de .gitignore protege la entrega y evita errores de push por archivos grandes.

El versionamiento es importante porque un workflow exportado es parte del código fuente del proyecto. Si una URL cambia, si se modifica un nodo Code o si se corrige una ruta, el cambio debe quedar respaldado. En un contexto institucional, esta práctica permite auditar la evolución del pipeline y restaurar una versión anterior si una modificación rompe el flujo.

XX. RIESGOS Y MITIGACIÓN

El primer riesgo es la variabilidad de las fuentes públicas. Un portal puede cambiar la ruta, el nombre del archivo o entregar HTML en lugar del recurso esperado. Para mitigar esto, el pipeline valida respuestas y, cuando es posible, identifica dinámicamente el archivo vigente.

El segundo riesgo es depender de una ejecución única. Para fuentes críticas, la arquitectura puede incorporar reintentos y uso de la última versión válida si existe. El respaldo no debe ocultar el error; la notificación debe indicar qué fuente falló y qué acción se tomó.

El tercer riesgo es la falta de evidencia. Si el flujo descarga un archivo pero no deja registro, el resultado queda incompleto desde el punto de vista de auditoría. Por eso los archivos

Cuadro IX: Matriz de cumplimiento del reto final.

Criterio	Cómo se cumple	Evidencia
Fuentes públicas	Flujos para BCE, INEC, Supercias y MINEDUC.	Capturas y JSON.
Periodicidad	Workflows diario, mensual, trimestral y anual.	Nombres en n8n.
Respaldo crudo	Cada rama guarda evidencia.	Carpetas.
Transformación	Nodos Code normalizan registros.	Output de nodos.
Persistencia	Nodos Oracle usan Upsert.	SELECT en Oracle.
Duplicados	Coincidencia por indicador y fecha.	Configuración Oracle.
Arquitectura	Diagrama C4 Nivel 2.	Figura C4.
Versionamiento	JSON subidos a GitHub.	Repositorio.
Seguridad	Wallet excluido.	.gitignore.

crudos, logs, consultas en Oracle y mensajes de Telegram se tratan como salidas formales del sistema.

El cuarto riesgo es la sobrecarga de memoria. Esto se observó especialmente en el flujo anual. La mitigación aplicada fue aumentar memoria y ejecutar por ramas. En una evolución

productiva se recomienda dividir ese flujo en subworkflows o jobs independientes por institución.

XXI. MATRIZ DE CUMPLIMIENTO

La matriz de cumplimiento relaciona requerimientos prácticos con evidencias. Esta matriz facilita la revisión del evaluador porque transforma la explicación general en criterios concretos. Además, permite detectar si falta una captura, un JSON exportado o una consulta en Oracle antes de la entrega final.

XXII. EVIDENCIAS VISUALES DE LOS FLUJOS

Las capturas de pantalla se conservan dentro del artículo porque permiten comprobar que la arquitectura C4 no quedó solo como propuesta, sino que fue materializada en workflows reales de n8n. Las imágenes muestran los triggers, ramas HTTP, nodos Code, respaldos de archivo, persistencia en Oracle y notificación por Telegram. Estas evidencias son relevantes para la defensa porque facilitan explicar el recorrido de cada flujo sin depender únicamente del texto.

XXIII. TABLAS DE CONTROL CONSERVADAS

Además de las capturas de los workflows, se conservan tablas e imágenes de control del documento original. Estas permiten relacionar el primer reto con el reto final, identificar campos de log, revisar periodicidades y sustentar la evolución desde una automatización académica hacia un pipeline institucional.

XXIV. RECOMENDACIONES DE OPERACIÓN FUTURA

La primera recomendación es separar el flujo anual en subworkflows por institución. Esta decisión reduciría el consumo de memoria, permitiría ejecutar solo la fuente que se necesita actualizar y facilitaría programar cada rama según el

Cuadro X: Recomendaciones para evolución del pipeline.

Área	Recomendación
Flujo anual	Dividir por institución o subworkflow.
Logging	Crear un módulo reutilizable de auditoría.
Base de datos	Separar staging, normalización y modelo analítico.
Monitoreo	Registrar duración, filas, tamaño y estado.
Seguridad	Mantener wallets y credenciales fuera de GitHub.
BI	Conectar Oracle con dashboards para uso institucional.

calendario real de publicación. Supercias, MINEDUC, INEC y BCE no necesariamente publican datos en la misma fecha, por lo que una separación fina mejoraría el mantenimiento.

La segunda recomendación es crear un subworkflow de logging. Actualmente, cada rama puede construir mensajes o registros propios. Un subworkflow común recibiría parámetros

como fuente, periodicidad, estado, ruta, número de registros y mensaje. Con ello se reduciría duplicación y se mantendría un formato único de auditoría para todos los flujos.

La tercera recomendación es separar datos crudos, datos normalizados y datos analíticos. En esta fase académica se utilizó una tabla consolidada flexible. En una versión institucional convendría usar tablas staging por fuente y procesos de transformación hacia tablas finales del tablero. Esta separación permitiría controlar mejor la calidad y el significado de cada indicador.

La cuarta recomendación es registrar tiempos reales de inicio y fin. Aunque se manejan tiempos promedio de ejecución, una versión futura debería capturar automáticamente duración, cantidad de filas, tamaño de archivo y resultado por rama. Estos datos permitirían evaluar rendimiento, detectar fuentes lentas y justificar aumentos de memoria o procesamiento por bloques.

XXV. GUÍA DE DEFENSA TÉCNICA

Durante la defensa, el flujo diario debe presentarse como el ejemplo más simple y verificable. Se debe explicar que el mismo dato obtenido por HTTP tiene dos usos: conservar evidencia cruda y construir un registro normalizado. La clave de esta sección es mostrar que una automatización pequeña también cumple el ciclo completo de RPA: activación, extracción, respaldo, transformación, persistencia y notificación.

El flujo mensual debe defenderse desde el manejo de archivos. Su importancia está en demostrar que el pipeline no solo consulta APIs, sino que también procesa documentos descargables. BCE IEE y Supercias Directorio exigen lectura estructurada y control de volumen. En esta parte se debe recalcar que el respaldo crudo permite reprocesar la fuente si cambia la lógica de transformación.

El flujo trimestral debe explicarse como la evidencia más fuerte de procesamiento dinámico. ENEMDU exige identificar el ZIP vigente, descargarlo, guardarlo, descomprimirlo y procesarlo. Este flujo demuestra que el sistema no depende únicamente de enlaces fijos y que puede adaptarse a publicaciones que cambian con cada periodo.

El flujo anual debe defenderse indicando que es el más pesado. La validación por ramas no representa una debilidad; es una estrategia razonable en un entorno local. En un ambiente productivo podría ejecutarse con más memoria o dividirse por institución. Lo importante es que la arquitectura anual muestra todas las fuentes y conserva el patrón común de extracción, respaldo, transformación y carga.

Al explicar Oracle, se debe mencionar que los cuatro flujos convergen en una estructura común. Esto demuestra integración y evita que el proyecto parezca una colección de automatizaciones aisladas. La operación Insert or Update permite ejecutar nuevamente un flujo sin duplicar registros,

porque la coincidencia se realiza por indicador y fecha fiscal. Al explicar Telegram, se debe resaltar la separación entre construcción del mensaje y envío. El nodo Code prepara contenido y el nodo Telegram ejecuta la comunicación. Esta separación hace que la notificación sea mantenible y permite agregar en el futuro conteos, errores parciales o enlaces a evidencias.

XXVI.

VALIDACIÓN SQL Y CONTROL DE DUPLICADOS

La validación en Oracle debe realizarse después de ejecutar cada workflow. No basta con observar que el nodo se muestre en verde dentro de n8n; también es necesario comprobar que los registros se encuentren disponibles en la tabla consolidada. Por eso se definieron consultas de revisión por indicador, conteo por fuente y detección de duplicados.

Para el flujo diario se recomienda filtrar indicadores como PRECIO_PETROLEO_WTI y RIESGO_PAIS. Esta consulta confirma que el flujo consultó las fuentes, transformó la respuesta y realizó la operación de persistencia. Para el flujo mensual se revisan los indicadores relacionados con BCE IEE y Supercias Directorio. En ENEMDU se validan los periodos trimestrales. En el anual se revisan registros provenientes de Supercias Ranking, MINEDUC, Censo y BCE.

El control de duplicados se apoya en la combinación INDICADOR y FECHA_FISCAL. Si el workflow se ejecuta dos veces para el mismo periodo, el resultado esperado no es duplicar filas, sino actualizar el registro existente. Esta decisión es importante porque un pipeline programado puede ejecutarse nuevamente por prueba, corrección o reintento. Sin Upsert, cada ejecución generaría ruido en la base y afectaría la calidad del tablero.

La revisión de FECHA_EXTRACCION también aporta valor. Permite saber cuándo fue procesado el dato, incluso si la fecha fiscal corresponde a un periodo anterior. Esta diferencia entre fecha fiscal y fecha de extracción es necesaria porque algunas fuentes publican información histórica o anual, pero el flujo puede cargarla en una fecha posterior.

El campo ESTADO permite distinguir registros completos, incompletos o con observación. Aunque la versión académica se concentra en cargas completas, conservar este campo prepara el sistema para escenarios de operación real. Si una fuente falla o se usa respaldo, el registro puede marcarse y revisarse posteriormente.

Finalmente, el campo DATOS_JSON brinda flexibilidad. Las fuentes no comparten exactamente las mismas columnas, por lo que una estructura rígida para todas podría ser difícil de mantener. Con JSON se conserva el contenido normalizado sin perder información específica. En una siguiente fase, estos registros podrían transformarse hacia tablas analíticas especializadas.

En la defensa, esta validación se puede explicar como una evidencia más fuerte que la captura visual del workflow. La captura muestra la estructura del proceso, pero la consulta SQL demuestra que el dato llegó a la base. Por eso el reto incluye ambas evidencias: el flujo visual de n8n y la persistencia verificable en Oracle Cloud.

El documento reducido centra el análisis en el reto final y conserva el primer pipeline como antecedente. Esta organización mejora la coherencia porque el marco conceptual, la

Componente	Función dentro del flujo
Inicio de Pipeline	Activa manualmente la ejecución del workflow.
Open-Meteo	Consulta clima por ciudad y calcula estadísticas regionales.
REST Countries	Obtiene indicadores básicos de Ecuador, Colombia y Perú.
Open Library	Filtra recursos académicos sobre automatización.
Merge + Reporte Final	Integra las tres ramas y genera una salida estructurada.

Figura 8: Componentes del primer pipeline y conceptos reutilizados en el reto final.

Campo	Propósito
version	Identificar la versión lógica del pipeline.
timestamp	Registrar fecha y hora de ejecución.
estado	Indicar si la ejecución fue completa o incompleta.
fuentes	Enumerar las APIs consultadas.
clima_regional	Guardar estadísticas climáticas agregadas.
contexto_pais	Guardar indicadores normalizados de países.
recursos_academicos	Guardar libros filtrados desde Open Library.
errores	Registrar fallos detectados por rama.

Figura 9: Campos de control trasladados del reporte JSON inicial hacia los logs del reto final.

Periodicidad	Fuentes	Salida propuesta
Diaria	BCE petróleo WTI y riesgo país	Tablas Oracle + log de ejecución
Mensual	BCE sector real y Supercias directorio	Esquema por fuente
Trimestral	INEC ENEMDU	Indicadores laborales normalizados
Anual/estática	BCE cuentas, Supercias ranking, MINEDUC y Censo	Datos históricos y registros base

Figura 10: Organización técnica de fuentes por periodicidad y salida propuesta.

arquitectura, la implementación y la validación se orientan al Pipeline RPA Macroentorno Ecuador.

El modelo C4 Nivel 2 fue adecuado para describir la arquitectura porque muestra contenedores y sistemas relacionados sin detallar cada nodo interno. De esta forma, el diagrama cumple una función arquitectónica y no solo operativa.

La separación de workflows por periodicidad fue una decisión correcta. Permite reducir carga, organizar mejor las

fuentes y programar cada flujo según la frecuencia real de actualización. El respaldo crudo es indispensable porque conserva evidencia de origen y permite reprocesar información si cambia la lógica de transformación.

La carga en Oracle Cloud convierte el reto de una simple automatización de descarga en una solución de datos más completa. Telegram agrega observabilidad operativa, porque el pipeline informa el cierre de ejecución sin depender de una

Aspecto	Primer reto	Reto final
Fuentes	APIs públicas: Open-Meteo, REST Countries y Open Library.	Instituciones públicas: BCE, INEC, Supercias y MINEDUC.
Activación	Manual Trigger.	Schedule Trigger por periodicidad.
Salida	Reporte JSON consolidado.	Carga en Oracle DB, trazabilidad y notificación.
Errores	Continuar salida regular y registrar error básico.	Reintentos, logs, alerta y control por fuente.
Alcance	Práctica académica multi-fuente.	Pipeline institucional para equipo de Datos.

Figura 11: Comparación entre el primer reto y el reto final aplicado al macroentorno.

Criterio	Pregunta de control
Disponibilidad	¿La fuente respondió correctamente?
Integridad	¿El archivo o respuesta contiene datos?
Formato	¿La extensión o estructura es la esperada?
Coherencia	¿Existen campos u hojas obligatorias?
Persistencia	¿Los datos se cargaron en Oracle DB?
Notificación	¿Se informó el resultado al equipo de Datos?

Figura 12: Criterios de control considerados para revisar disponibilidad, integridad y notificación.

Cuadro XI: Consultas de validación recomendadas.

Consulta	Objetivo	Resultado esperado
SELECT indicador	por Confirmar carga de cada fuente.	Registros recientes en Oracle.
COUNT indicador	por Medir volumen procesado.	Conteo coherente con la ejecución.
MAX fecha	Revisar última actualización.	Fecha de extracción reciente.
GROUP periodo	BY Detectar duplicados.	Cero registros repetidos.

Estado incompleto	Revisar incidentes.	Solo	casos justificados
-------------------	---------------------	------	--------------------

revisión manual del editor.

El flujo anual debe manejarse con especial cuidado por el tamaño de sus fuentes. La validación por ramas es una estrategia adecuada para ambientes locales con recursos limitados. En una versión productiva se recomienda separar subworkflows, mejorar logs, monitorear tiempos reales y conectar los resultados con tableros de Business Intelligence.

REFERENCIAS

- [1] R. Syed et al., "Robotic Process Automation: Contemporary themes and challenges," *Computers in Industry*, vol. 115, 2020.
- [2] J. Wewerka and M. Reichert, "Robotic Process Automation – A Systematic Literature Review and Assessment Framework," *arXiv:2012.11951*, 2020.
- [3] n8n, "Workflow automation platform and node documentation," n8n Docs, 2026.
- [4] Docker, "Volumes and bind mounts documentation," Docker Docs, 2026.
- [5] Oracle, "About Connecting to an Autonomous AI Database Instance," Oracle Documentation, 2026.
- [6] S. Brown, "The C4 model for visualising software architecture," C4Model.com, 2026.
- [7] DGTITD-UTPL, "Pipeline de extracción automatizada de datos públicos del macroentorno ecuatoriano: reto de 7 semanas," Universidad Técnica Particular de Loja, 2026.
- [8] GitHub, "About large files on GitHub," GitHub Docs, 2026.